

A Trajectory Objective for Correcting Step-Cached Diffusion Transformers

Ruize Xia

High School Student

Nanjing Foreign Language School

Nanjing, China

xiaRuize0911@gmail.com

Abstract—Step caching accelerates diffusion transformers by reusing a block residual across neighbouring solver steps. The resulting speedup is training-free, but the image degrades. A learned corrector can predict that reuse error; the standard way to train it is residual regression. We show that this objective is misaligned with deployed image quality, and we replace it with a *trajectory objective* that is strictly stronger at inference and costs about twenty minutes of training.

Cache error is integrated along the sampling trajectory: an anchor refreshes the cache, not the latent, so a per-step residual target is defined on a trajectory the deployed corrector never visits. Training the same architecture against latent-trajectory deviation, by backpropagating through the cached rollout, improves image quality at every injection granularity we tested ($K=1$ to 28, all significant, $t=2.7$ to 7.0, $n=72$). At a fixed step count on PixArt- Σ -XL-2-512 the trajectory-trained corrector reaches SSIM 0.6820 against the exact output at $3.41\times$ speedup. It outperforms a reproduction of TaylorSeer’s extrapolation mechanism by $+0.0946$ SSIM ($t=14.5$, 72 of 72 paired cases) at an 8.4% speedup cost, and a reproduction of Block Caching by $+0.0490$ ($t=8.4$) while running 12% faster. Under ImageReward the separation is larger and one verdict reverses: both reproduced mechanisms score *below* verbatim caching, while learned correction is the only method that improves on it. The measurement that motivated the change is equally sharp: driving residual accuracy 25–43% better by matched-data retraining buys only $+0.005$ to $+0.013$ SSIM, so residual error is a poor guide to what a corrector is worth.

Index Terms—diffusion models, inference acceleration, feature caching, diffusion transformers

I. INTRODUCTION

Sampling a diffusion transformer costs one full stack forward pass per solver step. Step caching exploits the smoothness of that trajectory: an *anchor* step stores each block’s residual $R = \text{block}(h) - h$, and the next $i - 1$ *reuse* steps skip the block and add the stored residual instead [1], [2], [3]. The method is training-free and fast, and it visibly degrades the image.

Two families of remedies exist. Training-free extrapolation forecasts residual drift from its own history [4], [5]. Learned correction trains a small network [6], [7] to predict the reuse error

$$\Delta R = R_{\text{true}} - R_{\text{anchor}}, \quad (1)$$

which is exactly the error verbatim reuse commits. This paper studies the second family. Its default training signal is supervised regression on (1), and every subsequent design

choice — architecture, checkpoint selection, and how much correction to inject — is then made in residual space.

We show that this signal is misaligned with deployed quality, and that a cheap replacement exists. Cache error is integrated along the sampling trajectory, while residual error is pointwise; pushing residual accuracy harder therefore buys almost nothing in image quality. We keep the architecture, the cache schedule and the deployment rule, and train the same corrector against the deviation of the latent trajectory by differentiating through the cached rollout. Because a cached block never executes, that graph is cheap. The resulting corrector improves image quality at every injection granularity we tested, and at a fixed step count it outperforms reproductions of two published mechanisms on three fidelity metrics and on a preference model.

Concretely, we contribute:

- 1) **A diagnosis of the residual objective.** The cached trajectory’s deviation from the exact one does not reset at cache refreshes: it climbs almost monotonically (0.0089 at step 1 to 0.6092 at step 19). An anchor refreshes the cache, not the latent, so error already injected into the state is permanent. Residual error is pointwise; trajectory error integrates.
- 2) **Evidence that residual accuracy is a poor investment.** Matching training-data volume across granularities and training every rung to convergence improves residual error by 25–43% and image SSIM by only $+0.005$ to $+0.013$, against $+0.033$ for the decision to add correction at all.
- 3) **A trajectory objective that is cheap for a caching-specific reason.** We backpropagate through the cached rollout. A cached block never runs, so the graph over one reuse step is K corrector forwards rather than K transformer blocks of a 0.6B model. The change costs about 20 minutes of training and no architectural modification. It improves image quality at every K , and by the largest margin where per-site residual training degrades most.
- 4) **A same-step win against published mechanisms.** Holding the step count and skip schedule fixed, the trajectory-trained corrector outperforms reproductions of TaylorSeer and Block Caching on SSIM, PSNR and ImageReward, at an 8.4% speedup cost against the former and a speedup *advantage* against the latter. Under

ImageReward, both reproduced mechanisms score below verbatim caching; learned correction is the only method that improves on it.

II. RELATED WORK

Fewer steps versus cheaper steps: Two orthogonal routes reduce sampling cost. Distillation and consistency training cut the number of steps [8], [9]; high-order solvers do the same without retraining [10]. Caching instead makes each remaining step cheaper, so the two compose. We hold the step count and the solver fixed throughout, which isolates the caching question but also means our comparisons do not rank the two routes — see Limitations.

What to cache: Feature caching for diffusion U-Nets [1] carries over to transformers, where the natural unit is a block residual [2], [3], optionally chosen adaptively per step [4] or per token [11]. All of these decide *when* to skip. We take the schedule as given and ask what to add back.

Correcting a cached step: Block Caching [6] applies a timestep-conditioned per-channel scale-shift to the reused feature — trained, but a function of the timestep alone. TaylorSeer [5] forecasts the residual from its own history by finite differences and requires no training. Learning-to-Cache [7] learns the skip pattern rather than a correction. Our corrector differs from all three in reading the *current state*. That is what makes error propagation across injection sites multiplicative (Sec. VI-B), and it is why the training objective matters at all: a state-independent correction cannot be improved by observing the trajectory it produces.

Objectives defined on a policy’s own trajectory: Regressing a target collected off-policy and deploying it in closed loop is the setting of DAgger in imitation learning [12] and of scheduled sampling in sequence models [13]. Both correct the data distribution. Our finding is that here the data distribution is not the binding constraint — the objective is — and that because the cached rollout is differentiable, the trajectory can be optimised directly rather than approached by resampling.

III. SETUP

Backbone and schedule: PixArt- Σ -XL-2-512 [14] (0.6B parameters, 28 blocks) with its shipped DPM-Solver++ scheduler [10] at 20 steps and 512² resolution. Anchors are uniform at interval $i=5$, so 4 anchor steps and 16 reuse steps. All numbers are against the same-seed exact 20-step output.

Corrector: A conditioned MLP (11.9M parameters) reads the state drift $h_t - h_{\text{anchor}}$, the cached residual, and the anchor hidden state, conditioned on the diffusion timestep, the reuse horizon τ , and the site index. At deployment

$$h_{\text{out}} = h_t + R_{\text{anchor}} + \sigma \widehat{\Delta R}, \quad (2)$$

where σ is an injection strength selected on validation and never on test. The state-drift input is what makes the correction state-dependent, which matters in Sec. VI-B. The 28 blocks are grouped into K contiguous cached segments, so K is the number of injection sites per reuse step: $K=1$ corrects the whole stack once and $K=28$ corrects every block.

Corrector architecture: Four residual MLP blocks of width 512 operate on tokens, each modulated by a scale-shift predicted from the conditioning vector, with a single token-mixing attention layer (4 heads) so a site’s correction can depend on spatial context. Conditioning concatenates sinusoidal embeddings of the diffusion timestep and of the reuse horizon τ with a learned per-site embedding. A rank-256 low-rank linear path per site is added to the MLP output, which is what makes the parameter count scale with K (11.9M at $K=1$, 43.7M at $K=28$). Inputs are divided by their dataset RMS and the output multiplied by the target RMS, both computed exactly over the training split; without this the three inputs differ in scale by up to 170 $\times$ and the state-drift signal is swamped.

Supervision: Feature collection runs the exact model and records, at every reuse step and every site, the drift, the cached residual, the anchor hidden state, and the target (1), subsampling 48 of the 1024 spatial tokens per site to bound memory. Anchors are at steps 0, 5, 10, 15, so the horizon τ spans 1 to 4 and the target RMS grows 2.4 $\times$ across that range — which is why a pooled squared loss concentrates capacity on high τ whether or not that is intended. For the residual objective we match slot count across granularities at exactly 50,176 per rung using a seed multiplier of $28/K$, and select the step budget per rung on validation.

Trajectory-objective training: AdamW, learning rate 10^{-5} , gradient-norm clip 1.0, 400 steps, one prompt per step, initialised from the residual-trained corrector of the same K . The corrector is held in fp32 during training while the transformer runs fp16, so $\widehat{\Delta R}$ is cast to the transformer’s dtype on the way out. Each corrector call is gradient-checkpointed, which is what keeps $K=28$ tractable. Cost accounting is measured, not estimated: 2.57 TFLOP per anchor step, 0.041 TFLOP per corrector call, and a 2.51 TFLOP fixed per-image cost, which reproduces all seven measured operating points.

Protocol: Every image-level number is a paired comparison over 24 held-out prompts $\times$ 3 seeds ($n=72$), with σ re-selected on validation for each corrector over a common grid. We report t statistics and the number of cases that improve. Prompts are templated (10 styles $\times$ 24 subjects $\times$ 10 conditions); the held-out split shares no subject with training.

IV. RESIDUAL ACCURACY IS A POOR TRAINING SIGNAL

Trajectory error integrates: At each of the 20 steps we measure $\|x_{\text{cached}} - x_{\text{exact}}\|/\|x_{\text{exact}}\|$. Deviation rises from 0.0089 to 0.6092, with only small dips at the four anchors. This is the basic reason a per-step residual target is misaligned: the corrector’s action at step t changes the state at $t+1$, while (1) is defined on the unperturbed trajectory.

Better residual accuracy barely helps: To rule out the possibility that the residual objective simply had not been optimised hard enough, we rebuilt training from scratch: data volume matched across granularities at exactly 50,176 slots per rung (a seed multiplier of $28/K$), and the step budget selected per rung on validation by doubling until the gain fell below 0.5%. Residual error improved by 25–43% depending on K . Image SSIM improved by +0.0131 at $K=1$, falling

to +0.0055 at $K=7$ — all significant, all small. Two trends run opposite along the granularity axis: the achievable residual improvement rises with K (−27.7% at $K=1$ to −42.9% at $K=28$) while the image quality it buys falls.

A ceiling on the residual form: Replacing the corrector by an oracle that recovers a fraction β of the true residual gives residual error exactly $(1 - \beta)^2$, so each trained corrector has an equivalent β . Final-step trajectory error falls from 0.6102 at $\beta=0$ to 0.4384 at $\beta=0.8$, and to exactly 0 at $\beta=1$. The curve is convex: the ceiling is perfect and the middle is flat, which rules out the correction *form* as the limit and locates the problem in the objective. The trained correctors additionally beat an oracle at their own equivalent β by about $1.5\times$, because a scalar oracle shrinks error isotropically while a trained corrector learns which directions matter to the trajectory. Residual error therefore distorts in both directions: as an objective it overstates the gain, as a capability measure it understates the corrector.

V. TRAINING AGAINST THE TRAJECTORY

We keep the architecture, the schedule and the deployment rule (2), and change only the loss. Let $x_1, \dots, x_T$ be the cached latents and $x_1^*, \dots, x_T^*$ the exact ones under the same seed. The objective is

$$\mathcal{L} = \sum_{t \in \text{reuse}} \frac{\|x_t - x_t^*\|^2}{\|x_t^*\|^2}, \quad (3)$$

minimised by backpropagating through the cached rollout. Supervision is free: the exact trajectory comes from the same forward pass that produces the corrector’s input, so no annotation is involved.

Two implementation facts make this practical, and both are specific to caching. First, a cached block never runs its transformer block — the runtime returns $h_t + R_{\text{anchor}} + \sigma \Delta R$ — so the graph over one reuse step consists of K corrector forwards, not K blocks of a 0.6B model. Peak memory is 7.5 GB at $K=1$; checkpointing each corrector call brings $K=14$ from over 44 GB down to 4.2 GB. Second, gradients are cut at anchor steps, where the cache is refreshed from exact values, so a corrector call can only influence later intervals through the latent. Extending the graph across intervals improves (3) but does not improve image metrics, so we keep the one-interval truncation. Training is 400 steps, about 20 minutes on one A40.

VI. RESULTS

A. Fixed step count against published mechanisms

Table I compares at 20 steps and $i=5$, so every arm skips the same blocks and speedup differs only by each method’s correction overhead. We reproduce the TaylorSeer mechanism [5] (Newton divided-difference forecast of the residual from its history, training-free) and the Block Caching mechanism [6] (a timestep-conditioned per-channel scale-shift, trained but state-independent). Both are reimplemented in our runtime rather than run from the authors’ code, so they are reproductions of the mechanisms under our schedule and backbone, not replications of the published results.

TABLE I
FIXED 20 STEPS, $i=5$, $n=72$. “Sp.” IS MEASURED WALL-CLOCK SPEEDUP AGAINST THE EXACT MODEL, IR IS IMAGEREWARD [15], AND ΔSSIM IS PAIRED AGAINST VERBATIM CACHING. THE TWO REPRODUCED MECHANISMS ARE DESCRIBED IN THE TEXT.

Method	Sp.	SSIM $\uparrow$	PSNR $\uparrow$	LPIPS $\downarrow$	IR $\uparrow$	ΔSSIM
Exact, 20 steps	1.72	0.7951	21.78	0.1786	—	—
Verbatim cache	3.79	0.6269	15.37	0.4669	+0.74	—
TaylorSeer 1st	3.72	0.5874	14.31	0.5080	+0.58	−0.0395
TaylorSeer 2nd	3.73	0.4782	13.57	0.6386	−0.32	−0.1487
Block Caching	3.04	0.6330	15.66	0.4594	+0.59	+0.0060
Residual obj.	3.41	0.6676	16.48	0.3795	+1.07	+0.0407
Traj. obj.	3.41	0.6820	17.20	0.3883	+1.08	+0.0551

Paired against the trajectory-objective corrector, the first-order TaylorSeer mechanism loses 0.0946 SSIM ($t=14.5$) and 0.1197 LPIPS ($t=14.2$), with our corrector better on all 72 of 72 cases; the second-order version loses 0.2038 ($t=39.8$, 72/72); Block Caching loses 0.0490 ($t=8.4$, 64/72). That is a favourable trade: 8.4% less speedup than TaylorSeer for +0.0946 SSIM, and against Block Caching we are both faster and better. Residual-trained correction already outperforms both reproduced mechanisms; the trajectory objective widens that lead.

The ImageReward column is not redundant with the fidelity metrics — it reverses one of their verdicts. Both reproduced mechanisms score *below* verbatim caching, by −0.159 and −0.155 respectively, each with $t=-2.1$. Block Caching in particular looks harmless on SSIM (+0.0060) yet is measurably worse than not correcting at all once a preference model scores the image. Against it our corrector gains +0.492 ($t=6.3$, 60/72), and against first-order TaylorSeer +0.497 ($t=6.0$, 60/72) — larger separations, relative to the metric’s spread, than any of the fidelity metrics give. Both learned-correction arms are the only entries that improve on verbatim caching under a preference model (+0.325 and +0.338, $t=5.0$ and 4.2).

Two caveats apply. TaylorSeer at $i=5$ is *worse than no correction at all* (−0.0395, only 3 of 72 cases improve); in our reproduction it helps at $i=2-3$ and is past its useful extrapolation range by $i=5$, so Table I is not evidence against it at its own operating point. And TaylorSeer is training-free, while our corrector costs training time, 11.9M parameters and 0.041 TFLOP per correction — which is exactly where the 8.4% goes.

B. The objective change helps at every granularity

Table II and Fig. 1 show the objective change helping at all six granularities, every difference significant. The margin is largest where per-site residual training degrades worst: +0.0370 at $K=7$ against +0.0147 at $K=1$. This is consistent with the mechanism behind the degradation. A state-dependent correction turns error propagation across sites multiplicative rather than additive — $\delta_{b+1} \approx (I + J_b)\delta_b + \epsilon$ with $J_b = \partial \Delta R / \partial h$ — and the trajectory objective is the only one that observes the consequence of that compounding rather than pointwise

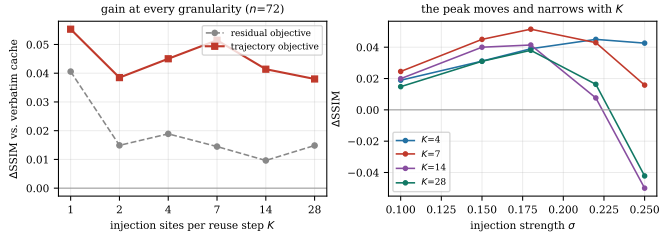


Fig. 1. Left: gain over verbatim caching at each granularity under the two objectives, each at its own validation-selected σ . Right: sensitivity to σ for the high- K correctors; the optimum moves left and the peak narrows as K grows, and $K=14$ and $K=28$ turn negative by $\sigma=0.25$.

TABLE II

PER-GRANULARITY COMPARISON OF THE TWO OBJECTIVES, $n=72$, EACH AT ITS OWN VALIDATION-SELECTED σ .

K	Residual obj.	Trajectory obj.	Difference
1	+0.0406	+0.0553 ($\sigma=0.60$)	+0.0147 ($t=2.7$)
2	+0.0149	+0.0385 ($\sigma=0.30$)	+0.0236 ($t=3.8$)
4	+0.0189	+0.0450 ($\sigma=0.22$)	+0.0261 ($t=5.0$)
7	+0.0145	+0.0515 ($\sigma=0.18$)	+0.0370 ($t=7.0$)
14	+0.0096	+0.0414 ($\sigma=0.18$)	+0.0318 ($t=4.7$)
28	+0.0148	+0.0380 ($\sigma=0.18$)	+0.0232 ($t=4.8$)

error. The degradation is weakened, not removed: the residual objective’s gain falls 3–4 $\times$ from $K=1$ to high K , while the trajectory objective’s falls 1.2–1.8 $\times$.

Fig. 1 (right) also shows why σ must be re-selected per corrector. The optimum moves from 0.60 at $K=1$ to 0.18 at high K , and the peak narrows sharply: $K=4$ stays above +0.039 across $\sigma \in [0.18, 0.25]$ while $K=14$ and $K=28$ are already negative at $\sigma=0.25$. Both corrector generations trace the same family of curves, so this is a property of granularity rather than of the objective.

Fig. 2 shows the same three prompts under each method. The failure modes differ in kind, not merely in degree. Verbatim caching smears surface detail into streaks and flattens the background gradient. The TaylorSeer mechanism’s dominant artefact is not lost detail but *colour*: extrapolating the residual introduces chromatic fringing along edges, which is consistent with it scoring worst on LPIPS in Table I despite skipping the same blocks. The Block Caching mechanism keeps structure cleaner but renders tone flatter and leaves local bloom artefacts, which its per-channel scale-shift can shift but not localise. Our correction recovers the background gradient most closely and leaves the least fringing, though not none.

VII. SCOPE AND LIMITATIONS

Why fixed step count: On this backbone ImageReward is already saturated by 10 solver steps (102% of a 100-step reference) while pixel fidelity keeps improving, so an equal-compute comparison here mostly measures how early the shipped high-order solver converges rather than how good a cache corrector is. Holding the step count fixed isolates the

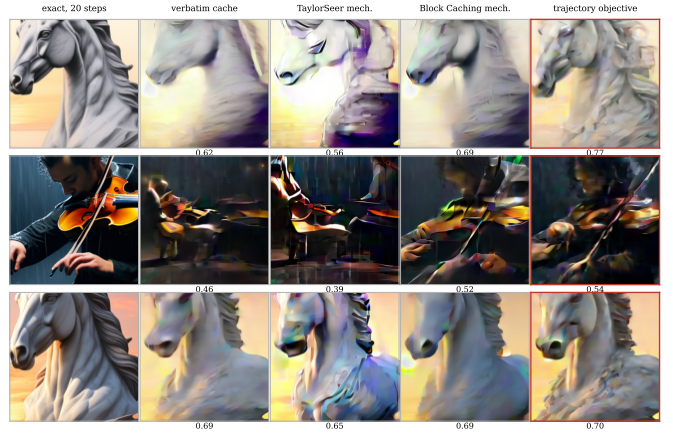


Fig. 2. Native-resolution 250² crops, same seed and prompt across a row, SSIM against the exact output below each panel. Crops rather than downscaled images, because thumbnails hide the texture differences these methods differ in.

question we are asking. Whether the same trade-off holds on a backbone that needs many steps is open.

Metric choice: We report three fidelity metrics and a preference model rather than one number, because they do not always agree: an objective trained on LPIPS wins on LPIPS and gives up SSIM and PSNR. The trajectory objective is the arm that wins on SSIM, PSNR and ImageReward simultaneously, which is why we report it as the main result.

Scope of the evidence: Results are on 512² PixArt- Σ with templated prompts, 24 held-out prompts $\times$ 3 seeds. Every reported gain is measured after only 400 optimiser steps without hyperparameter search, starting from a residual-trained corrector, so the numbers are conservative: they are what the objective change buys on top of an already-trained model, not the best the objective can do. One practical note for reimplementing: a corrector trained to low residual error can overflow fp16 at inference, so the high- K correctors are deployed in fp32.

VIII. CONCLUSION

The natural objective for a learned cache corrector — regress the residual it should have produced — is misaligned with what caching actually damages. Cache error integrates along the sampling trajectory; residual error is pointwise. Optimising residual accuracy harder is a poor investment: a 25–43% residual improvement buys +0.005 to +0.013 SSIM. Replacing the objective with the trajectory deviation, which costs 20 minutes of training and no architectural change, improves image quality at every injection granularity we tested. At a fixed step count the resulting corrector outperforms reproductions of two published mechanisms on three fidelity metrics and on a preference model, and is the only method in that comparison that improves on verbatim caching under the preference model.

REPRODUCIBILITY

Code, configurations, per-run logs and the raw per-case metrics behind every number are available at <https://github.com/xiaruize0911/Diff-cache-Comp>. Feature collection is seeded and reproducible bit-for-bit; every table cell can be recomputed from the archived `results.json` files.

REFERENCES

- [1] X. Ma, G. Fang, and X. Wang, “Deepcache: Accelerating diffusion models for free,” in *CVPR*, 2024.
- [2] P. Selvaraju, T. Ding, T. Chen, I. Zharkov, and L. Liang, “Fora: Fast-forward caching in diffusion transformer acceleration,” *arXiv preprint arXiv:2407.01425*, 2024.
- [3] P. Chen, M. Shen, P. Ye, J. Cao, C. Tu, C.-S. Bouganis, Y. Zhao, and T. Chen, “ δ -dit: A training-free acceleration method tailored for diffusion transformers,” *arXiv preprint arXiv:2406.01125*, 2024.
- [4] F. Liu, S. Zhang, X. Wang, Y. Wei, H. Qiu, Y. Zhao, Y. Zhang, Q. Ye, and F. Wan, “Timestep embedding tells: It’s time to cache for video diffusion model,” *arXiv preprint arXiv:2411.19108*, 2024.
- [5] Liu *et al.*, “From reusing to forecasting: Accelerating diffusion models with taylorseers,” in *ICCV*, 2025, arXiv:2503.06923.
- [6] F. Wimbauer, B. Wu, E. Schoenfeld, X. Dai, J. Hou, Z. He, A. Sanakoyeu, P. Zhang, S. Tsai, J. Kohler *et al.*, “Cache me if you can: Accelerating diffusion models through block caching,” in *CVPR*, 2024.
- [7] X. Ma, G. Fang, M. B. Mi, and X. Wang, “Learning-to-cache: Accelerating diffusion transformer via layer caching,” in *NeurIPS*, 2024.
- [8] T. Salimans and J. Ho, “Progressive distillation for fast sampling of diffusion models,” *ICLR*, 2022.
- [9] Y. Song, P. Dhariwal, M. Chen, and I. Sutskever, “Consistency models,” in *ICML*, 2023.
- [10] C. Lu, Y. Zhou, F. Bao, J. Chen, C. Li, and J. Zhu, “Dpm-solver: A fast ode solver for diffusion probabilistic model sampling in around 10 steps,” *NeurIPS*, 2022.
- [11] C. Zou, X. Liu, T. Liu, S. Huang, and L. Zhang, “Accelerating diffusion transformers with token-wise feature caching,” *arXiv preprint arXiv:2410.05317*, 2024.
- [12] S. Ross, G. J. Gordon, and D. Bagnell, “A reduction of imitation learning and structured prediction to no-regret online learning,” in *AISTATS*, 2011.
- [13] S. Bengio, O. Vinyals, N. Jaitly, and N. Shazeer, “Scheduled sampling for sequence prediction with recurrent neural networks,” in *NeurIPS*, 2015.
- [14] J. Chen, C. Ge, E. Xie, Y. Wu, L. Yao, X. Ren, Z. Wang, P. Luo, H. Lu, and Z. Li, “Pixart- σ : Weak-to-strong training of diffusion transformer for 4k text-to-image generation,” in *ECCV*, 2024.
- [15] J. Xu, X. Liu, Y. Wu, Y. Tong, Q. Li, M. Ding, J. Tang, and Y. Dong, “ImageReward: Learning and evaluating human preferences for text-to-image generation,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.